

Project 8 - Online Car Rent

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overview

An “Online Car Rent” system that helps customers rent cars for different purposes. Two types of users: **ADMIN** and **MEMBER**.

- **ADMIN:** main controller – posts car availability and cost information, deletes any existing member, views all rent order history with related users, deletes anyone’s blog post.
- **MEMBER:** registered user – browses cars by categories, views car information and costs, places orders, sees invoice with cancel/finalize options, chooses payment method, views rental history in profile, posts blog experiences, deletes own blog posts.

Team Structure & Student IDs

Student ID	Task	Main Features
23-54051-3	Task 1	User Authentication (Admin/Member), Profile Management, Home Page with Featured Cars, Category Browsing (basic view)
23-54040-3	Task 2	Admin – Car Management (CRUD), Delete Members, View All Rent Order History
19-39469-1	Task 3	Member – Car Selection, Order Placement, Invoice (Cancel/Finalize), Payment Method Selection, Rental History in Profile
24-57937-2	Task 4	Blog Page – Post Experience, View All Blogs, Delete Own Post (Member) and Delete Any Post (Admin)

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/member), profile_picture, address, phone, created_at
cars	id, name, model, type (Private car/Microbus/Pick-up etc.), price_per_day, availability_status, image_path, description, created_at
orders	id, user_id (member), car_id, start_date, end_date, total_cost, status (pending/confirmed/cancelled), payment_method, order_date
payments	id, order_id, amount, payment_method (credit_card, bkaash, nagad, bank_transfer, etc.), transaction_id, payment_date
rentals (rental_history)	id, user_id, car_id, order_id, rental_date, return_date, total_cost (view – can be derived from orders) – or simply use orders table with status='confirmed'
blogs	id, user_id, title, content, created_at, updated_at

Note: rental_history can be replaced by querying orders with status='confirmed'. For clarity, keep orders as the main history source.

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with password_hash() / verified with password_verify()
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- session_start() on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return Content-Type: application/json; handle errors client-side
- File uploads go to public/uploads/ (profile pictures, car images) – validate MIME type + size server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.
4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, car posting, order, blog).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into `main` via PR.

Git Flow

- Repository has a `main` branch (protected, no direct pushes).
- Each student creates a **feature branch** from `main` named `feature/taskX-studentID` (e.g., `feature/task1-2350723-1`).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into `main` and merge.
- Final `main` must contain all four feature branches merged with full history.

Task Breakdown

Task 1 – User Authentication (Admin/Member), Profile Management, Home Page with Featured Cars, Category Browsing

Student: 23-54051-3

CRUD included: Create (registration), Read/Update (profile).

Requirements:

- **Registration** – form for ADMIN and MEMBER. Collect: name, email, password (≥ 8 chars), address, phone. Role dropdown (admin/member). Validate unique email, hash password. Redirect to login.

- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” cookie + token for auto-login.
- **Profile Page** (session-gated) – both admin and member can update name, email, address, phone, profile picture, change password (require current). Show success banner.
- **Logout & Navbar** – destroy session and cookie. Navbar shows links based on role.
- **Home Page** (after login, first page member sees) – shows:
 - Featured cars (some random or most rented cars).
 - Category bar (Private car, Microbus, Pick-up, etc.) – fetched from distinct car types.
 - Other navigation options (Blog, Order History, etc.).
- **Browse by Categories** – clicking a category shows all cars of that type (view-only for members). Car cards show name, model, price per day, image. Click leads to car details page (for ordering – Task 3).
- **JS validation** on registration and profile forms.
- **AJAX** – optional for dynamic category content loading or live car search.

Key outputs: `users` table with admin and member accounts; session management; home page with featured cars and category browsing.

Task 2 – Admin: Car Management (CRUD), Delete Members, View All Rent Order History

Student: 23-54040-3

CRUD included: Full CRUD on `cars` (Create, Read, Update, Delete) + Delete on `users` (members) + Read on `orders`.

Requirements:

- **Admin Gate** – every admin page checks `$_SESSION['role'] === 'admin'`; redirect others.
- **Car Management** – create, edit, delete car listings.
 - Form: name, model, type (dropdown: Private car, Microbus, Pick-up, etc.), price_per_day (>0), description, image upload (JPEG/PNG ≤2MB to `public/uploads/cars/`), availability status.
 - Edit pre-fills form.
 - Delete removes image file and car record (cascade to orders? Block if orders exist or set null – decide: block deletion if car has pending/confirmed orders).
- **Delete Member** – list all members (role='member') with delete button. Delete user and cascade their orders, blogs, etc. (or set foreign key to null).

- **View All Rent Order History** – separate admin page showing all orders with: member name, car details, rental dates, total cost, status, payment method. Filter by date or status.
- **Admin Dashboard** – summary counts (total cars, total members, total orders, total blog posts).
- **PHP & JS validation** on car forms (price positive, image type).
- **AJAX** – for delete member confirmation, inline order status update (optional).

Key outputs: Admin fully manages car inventory, removes members, views complete order history.

Task 3 – Member: Car Selection, Order Placement, Invoice (Cancel/Finalize), Payment Method Selection, Rental History in Profile

Student: 19-39469-1

CRUD included: Create (order), Read (rental history), Update (order status: cancel/finalize).

Requirements:

- **Member Gate** – only logged-in members (role='member') can order; browsing is public.
- **Car Details & Selection** – from car listing, member views details and selects a car.
- **Order Placement** – form: select start date, end date (validate dates). System calculates total cost (price_per_day * number of days).
- **Invoice Page** – after order submission, show invoice with: car details, rental period, total cost, and two buttons: **Cancel** and **Finalize**.
 - **Cancel** – deletes the order (or updates status to 'cancelled'), redirects to home.
 - **Finalize** – proceeds to payment section.
- **Payment Section** – after finalizing, member chooses payment method. Include all currently available payment systems: Credit Card, bKash, Nagad, Bank Transfer, Cash on Delivery (optional).
 - On payment confirmation, update order status to 'confirmed', store payment details in payments table.
 - Redirect to a success page with order summary.
- **Rental History in Profile** – in member's profile page, show all past orders (confirmed and cancelled) with car name, dates, total cost, status.
- **JS validation** on order form (dates not in past, start date before end date).
- **AJAX** – for calculating total cost dynamically when dates change, for cancelling order without page reload.

Key outputs: Members can rent cars, see invoice, choose payment method, view rental history. Orders and payments stored.

Task 4 – Blog Page: Post Experience, View All Blogs, Delete Own Post (Member) and Delete Any Post (Admin)

Student: 24-57937-2

CRUD included: Create (blog post), Read (view blogs), Delete (own post for member, any post for admin). Update not required but can be added.

Requirements:

- **Blog Page** – navigable from home page. Shows all blog posts in a block manner (title, author, content, date). Both ADMIN and MEMBER can view all posts.
- **Post a Blog** – logged-in members (and admin) can post their experience with the site.
 - Form: title, content (textarea).
 - Submit via AJAX or regular POST – saves to `blogs` with `user_id`.
- **Delete Own Post** – member sees a delete button next to their own posts. Clicking deletes the post (AJAX DELETE to `/api/blogs/{id}`).
- **Admin Delete Any Post** – admin sees delete buttons on all blog posts. Can delete anyone's post (AJAX or POST).
- **No comment section** on blogs (as per requirement).
- **JS validation** on blog post form (non-empty title and content).
- **AJAX** – for posting blog (asynchronous), deleting blog, and refreshing blog list without reload.
- **PHP validation** – ensure user is logged in, content sanitized, proper authorization for delete (member checks ownership, admin bypasses).

Key outputs: Fully functional blog page where members share experiences, delete their own, and admin moderates all posts. All CRUD operations on `blogs` table (Create, Read, Delete).